This is the same set of rules as for functions, except that there's no return value, so no `return` statement. Instead, the constructor initializes all the bases and data members in the member initialization list. Trivial copy constructors are implicitly `constexpr`.

A.4.4. constexpr and templates

When `constexpr` is applied to a function template, or to a member function of a class template, it's ignored if the parameters and return types of a particular instantiation of the template aren't literal types. This allows you to write function templates that are `constexpr` if the type of the template parameters is appropriate and just plain `inline` functions otherwise, for example:

```
template<typename T>
constexpr T sum(T a,T b)
{
    return a+b;
}
constexpr int i=sum(3,42);
std::string s=
    sum(std::string("hello"),
        std::string(" world"));
```

OK, sum<int> is constexpr

OK, but sum<std::string> isn't constexpr

The function must satisfy all the other requirements for a `constexpr` function. You can't declare a function with multiple statements `constexpr` just because it's a function template; that's still a compilation error.

A.5. Lambda functions

Lambda functions are one of the most exciting features of the C++11 Standard, because they have the potential to greatly simplify code and eliminate much of the boilerplate associated with writing callable objects. The C++11 lambda function syntax allows a function to be defined at the point where it's needed in another expression. This works well for things like predicates provided to the wait functions of `std::condition_variable` (as in the example in section 4.1.1), because it allows the semantics to be quickly expressed in terms of the